Exercise 7: An Agent Team Builds Your Frontend

Session: 7 — Agentic Workflows
Duration: 35 minutes, plus a 5-minute closing round
Project: `web`, the frontend you specified in Exercise 6.

Goal

In Exercise 6 you wrote a spec, a plan and a task list, and no code. In this exercise an agent team builds the code.

The teammates in an agent team can send messages to each other, without the lead. You use that here. One teammate reviews the work of the others. It sends what it finds straight to the teammate that wrote the code.

You do five things:

1. Build the shared part of the frontend, in one Claude session.
2. Write one prompt that starts a team: one builder per user story and one reviewer. Improve that prompt with a coach.
3. Start the team and watch it work.
4. Follow one finding from the reviewer to the builder, and check the result in the browser.
5. Let three subagents design a new look for your frontend, each in its own worktree.

You will not write application code yourself.

Start here

- **Terminal 1**, in `bookstore`: `go run .`
- **Terminal 2**, in `web` (inside `bookstore`): `claude`. This is your **lead** session for the whole exercise.
- **Terminal 3**, in `web`. Leave it open: you start the frontend there in task 1.
- You need the task list from Exercise 6. No task list? Task 1 gives you one.
- Training mode is off in `web`: Claude answers directly. Stuck? Type *just tell me*.

Tasks

1. Build the foundation, one session

You produce the **foundation**: Phase 1 and Phase 2 of `tasks.md`. These are the files every story needs, such as the API client, the page shell and the stylesheet. The constitution says no story may start before the foundation is done, so the lead builds it alone.

1. **Check your spec.** Your own spec from Exercise 6 is complete if `specs/001-*/tasks.md` exists and has at most three user stories. If it is complete, go to step 2. If not, or if you are not sure, use the reference spec. Paste this into the lead:

   > Copy the folder `specs-reference/001-browse-books` to `specs/001-browse-books`. If `specs/` already holds another folder, move that folder to `specs-old/` first. Then commit with the message "spec: reference spec for exercise 7".

2. Open `specs/001-*/tasks.md` and find **Phase 1** and **Phase 2**.

3. Send the lead this prompt. Copy it as written:

   > Read `specs/001-*/tasks.md` and `.specify/memory/constitution.md`. First fix the task list. If a user-story task writes a file that more than one story needs (page shell, stylesheet, API client, router), or two stories write the same file, move that work into Phase 2. Give each story its own page file, created as a stub in Phase 2, so no story edits the router. Tell me what you moved. Then implement Phase 1 and Phase 2, and nothing from any user story. Tick each task off in `tasks.md` when it is done. Then commit everything with the message "foundation" and add the git tag `foundation`. When you finish, list the files you created, and for each story the one file it owns.

   The run takes one to three minutes. Wait for it. While you wait, read the introduction of task 2. When the lead reports back, write down its file list. Task 2 needs it.

4. When the lead reports back, start the frontend in terminal 3:

```
npm run dev
```

   Open http://localhost:5173. You should see the page shell, and no errors in the browser console.

Why step 3 asks for this: if two stories need the same file, such as the router, two builders would edit it at the same time. Step 3 moves that work into the foundation before any code exists. You solve the problem in the plan, not later in the code. The tag `foundation` marks where the story work starts. The reviewer and the check in task 4 compare against it.

Done when: Phase 1 and 2 are ticked in `tasks.md`, the page shell loads on port 5173, and the lead reports the commit and the tag `foundation`.

2. Draft the team prompt

You produce one prompt that turns the lead into a team lead. This is the core of the exercise.

The lead starts **teammates**: separate Claude Code sessions. Together they are the **agent team**. A teammate starts with an empty context and does not see this chat. Everything it must know is in your prompt, or in a file your prompt names.

Your team has two kinds of teammates:

- A **builder** for each user story. It writes the code for that story.
- One **reviewer**. It writes no code. It checks each story and tells the builder what to fix.

A builder and the reviewer send messages to each other directly, by name. The lead does not pass the messages on. A subagent is different: it reports back only to the session that started it.

The reviewer also checks two **house rules**: rules your team agreed on that are not in the spec.

- Every page sets its own browser tab title.
- Everything you can click is a real link or button, so it works with the keyboard.

Only the reviewer gets these rules, so it will likely find something to send back. That is normal in a team: the spec says what to build, and the review checks how it is built.

Each user story in `tasks.md` has an **Independent Test**. It lists the steps from `quickstart.md` that must pass for that story. The example prompt below uses this test.

1. **Draft**. Open `tasks.md` and find the section **Parallel Team Strategy** near the bottom. It says how to split the work. Your prompt turns that plan into instructions the lead can follow. Start from the worked example below. It is not complete: the coach tells you what is missing. Words in `<angle brackets>` are placeholders. Replace each one with a name from your own `tasks.md`, and remove the brackets. Do not run the prompt yet.

   > Read `specs/<your spec folder>/tasks.md`. Spawn an agent team. Spawn one builder per user story, named after the story: `<us1-short-name>`, `<us2-short-name>`. Each builder works only on the tasks tagged with its story. Also spawn a teammate named `reviewer` that writes no code. When a builder's Independent Test in `tasks.md` passes against the running backend, the builder sends `reviewer` the list of files it changed. The reviewer first checks our two house rules. One: every page sets its own browser tab title, such as "Book list · BookStore". Two: everything you can click is a real link or button, so it works with the keyboard. Then it checks three more things: no foundation file changed (`git diff --stat foundation`), no backend address is in the code (`grep -rn "8080\|localhost" src/ index.html`), and every error path shows a fixed sentence, as principle V in `.specify/memory/constitution.md` says. Report per story: files changed, and what the reviewer found.

2. **Coach and revise**. Run `/parallel-coach 2` and paste your draft as your next message. The coach points out gaps. It never writes the prompt for you. Revise until the coach says the prompt is ready. Disagree with the coach? Say "*run it anyway*". It will let you, and tell you what to watch for.

Done when: the coach says your team prompt is ready, and task 1 is done.

3. Start the team and watch it

You start the team. Then you only watch. In this task you type nothing, except in the two cases at the end.

1. Paste your prompt into the lead, in terminal 2. The coach does not run this one for you. A team can only start from the session you type in.

2. Look at the agent panel below the prompt input. Within a minute you should see one row per teammate: each builder and the reviewer, with the names from your prompt. Rows without your names are plain subagents, not a team. In that case, tell the lead: "*Use an agent team, not subagents.*" Still no team? Agent teams are experimental. Tell the trainer, do not spend more time on it, and go on with task 5.
3. Watch. Keep `tasks.md` open in your editor: ticks appear as builders finish tasks. Refresh http://localhost:5173 now and then. Stories appear while you watch. To see what one teammate does, select its row with the **up and down arrows** and press **Enter**. To leave, select the lead's row again. Do not press **Escape** in a transcript: that interrupts the teammate.

You type something only in these two cases:

- A permission prompt from a teammate appears in the **lead's** row. Answer it there.
- The lead starts editing story files itself, or fixes a finding of the reviewer itself. Tell it: "*Wait for your teammates to finish.*"

Done when: the lead's report says the reviewer approved every story, every story task in `tasks.md` is ticked, and both stories show in the browser.

4. Follow one finding, then check it yourself

The teammates still exist after the run, each with its own memory of what it did. You follow one finding from the reviewer to the builder that fixed it. Most findings are about the two house rules from task 2: a tab title per page, and links and buttons that work with the keyboard. Then you check the result in the browser, which the reviewer did not open.

1. **Bring back the reviewer.** When no teammate is working, their rows disappear from the panel after a short time. They are not stopped. Tell the lead:

   > Ask reviewer to list every finding it sent, and to which builder.

   The reviewer's row comes back while it works. Select it with the arrows and press **Enter**. The row stays while you are in its transcript.

2. **Read and ask.** Scroll up in the reviewer's transcript. Find a message from a builder: its list of files. Then find what the reviewer sent back to that builder, for example a page without its own tab title. Now ask the reviewer a question yourself. Type:

> Which builder fixed what you sent back? How did you check the fix?

This message goes to the reviewer only. The lead does not see it. When you have the answer, select the lead's row again. Did the reviewer approve every story the first time? Then ask it how it checked the house rules, and skip step 3.

3. **Ask the builder.** Tell the lead:

> Ask `<builder>` what it changed after the reviewer's message, and show me its answer.

The builder still knows the finding. The lead never saw it.

4. **Check the house rules in the browser.** Click through each page and look at the browser tab: does the title change per page? Then click once in the page and press **Tab** a few times. You should reach every link and button, and see which one is selected.

5. **Quick check against the spec.** Follow the steps in `specs/001-*/quickstart.md`. Then test the three gaps from Exercise 6:

   - Go to page 2 of the book list. Does it show different books?
   - Go past the last page. What does an empty page show?
   - Open a book id that does not exist, such as `#/books/9999`. Do you see a fixed sentence, or raw error text from the backend?

6. **Run the check**: type `/verify-exercise 7`. Paste your team prompt when it asks. It grades the prompt first. Then it checks the files, including the checks the reviewer did. Bring its report to the closing round.

7. **Commit.** Tell the lead:

> Commit everything with the message "feat: bookstore-web, built by agent team".

Done when:

- you followed one finding from the reviewer to the builder
- you checked the house rules and the quickstart in the browser
- `/verify-exercise 7` has run, and the lead has committed

5. Three designs, three worktrees

The frontend works, but it looks plain. Three subagents now give it a new look, each a different one, at the same time. Each subagent works in its own **worktree**: its own branch and folder, so it cannot edit the files of the others. The three do not need to talk to each other. That is why you use subagents here, not an agent team.

1. Send the lead this prompt. Replace the three styles with your own ideas if you like:

   > Start three subagents, not teammates, each in its own worktree. Each one gives the frontend in `web` a new look: one "retro terminal", one "cosy bookshop", one "brutalist". They may change the stylesheet and the HTML markup only. They must not change the API client or what any page does. Each one commits its work on its own branch. When all three are done, list the three branch names.

2. **Look at each design.** For each branch, tell the lead:

   > Check out `<branch>` in detached mode.

   Detached means you look at that branch without switching your own branch. Then refresh http://localhost:5173. The dev server shows that design.

3. **Merge your favourite.** Tell the lead:

   > Switch back to the branch you started on, and merge `<favourite>`.

4. **Merge a second one.** Tell the lead:

   > Now also merge `<second branch>`. If there is a conflict, do not solve it. Show me the files with a conflict.

   Read what the lead reports. Then tell it: "*Abort that merge.*" No conflict? Then the two designs changed different files. Tell the trainer.

While the subagents worked, the worktrees kept them out of each other's files. The conflict came only when you merged two designs that changed the same file.

Done when: you have seen all three designs, your favourite is merged, and you have seen what the second merge does.

Closing round

The trainer asks people at random. Have answers ready:

- What did the coach say was missing in your team prompt before it said the prompt was ready?
- What did the reviewer send back? What did the builder change? Did it hold up in the browser?
- In Exercise 6 you marked some answers as guesses. Find one in the running frontend. Does the code treat it as a fact?
- Task 3 used an agent team. Task 5 used subagents in worktrees. Why the difference?
- Close with **one take-away**. Which part of this exercise needed teammates that talk to each other? Which part would have been faster in a single session?

What you should have

```
web/
  index.html, styles.css, src/     built from your spec
  specs/001-*/tasks.md             every story task ticked
  git log                          "spec" → "foundation" → "feat" → your design
```

Every line of application code in this project was written from a spec you wrote. You typed none of it yourself.

Before Session 8: think of one real team that you know well, with one codebase. Exercise 8 is about that team. You need no laptop project.